

Juggling with Pattern Matching

Jean Cardinal,¹ Steve Kremer,² and Stefan Langerman^{1,*}

¹Computer Science Department, Université Libre de Bruxelles,
B-1050 Brussels, Belgium
{jcardin,slanger}@ulb.ac.be

²École Normale Supérieure de Cachan
and INRIA Futurs,
61 avenue du Président Wilson,
94235 Cachan Cedex, France
kremer@lsv.ens-cachan.fr

Abstract. In the late 1980s it was shown that juggling patterns can be described by strings of numbers with fascinating combinatorial properties that have since then been studied by many mathematicians and computer scientists. In this paper we propose to study juggling patterns from a pattern matching point of view. Inspired by pattern matching algorithms based on convolution, we propose a fast algorithm for finding transitions between juggling states. Apart from being a fun application of pattern matching theory, it provides a practical tool in the experimental design of (large) juggling patterns. We also provide a compact formula for counting the number of transitions of a given length between two juggling states.

1. Introduction

Juggling can be defined as the art of tossing objects in the air and catching them, keeping several in the air at the same time. Different kinds of objects can be used: balls, sticks, clubs, rings, torches, etc., and juggling acts can involve more than one juggler. Objects are usually thrown according to some kind of regular pattern, the beauty and complexity of which make juggling popular among mathematicians, physicists, and, quite naturally, computer scientists.

* Chercheur qualifié FNRS.

There exist many scientific papers about juggling. The earliest ones are probably from Claude Elwood Shannon, the inventor of information theory, in the early 1950s [13], [14]. Among other results, he derived a physical law relating the time an object spends in the air and the time it spends in the hands of the juggler. He even designed a simple mechanical juggling machine.

An actual mathematical theory of juggling was however not available before the invention, by a number of different jugglers, of the *siteswap* notation, around 1985 [9], [15]. In this notation, a two-hand juggling pattern is denoted by a string of positive integer numbers, where each number represents a throw, the value of the number is related to the height of the throw, and the length of the string is equal to the period of the juggled pattern. This notation assumes that the throws are made alternately by each hand and that no two balls can be caught or thrown at the same time. A large number of contributions have already been made in the analysis of siteswap patterns. The number of patterns with b balls and a bounded throw height, for instance, was established and generalized by different authors [16], [2], [3]. We summarize the essential properties of siteswaps in Section 2.1.

In Section 2.2 we define the *landing state graph*, and explain how periodic juggling sequences correspond to loops in this infinite graph. These background notions can be found in the recent book by Polster [12], probably the most complete available source of information on mathematical aspects of juggling.

Our contribution is about finding transitions between juggling patterns, i.e., sequences of throws that allow movement from one pattern to another. This kind of problem was tackled only once in [10], where heuristic algorithms—neural networks and genetic algorithms—were used to find “nice” transitions.

It is not always possible to find a transition sequence of a given length. So our first concern will be to find what are the achievable lengths. In Section 3 we describe the problem of finding all achievable transition sequence lengths between two juggling patterns as that of finding paths in the landing state graph. We also show that it can be seen as a kind of pattern matching problem, where the destination landing state has to be suitably matched with the source landing state. A trivial algorithm with quadratic complexity is derived quickly. In Section 4 we review a recent variation from Kalai [6] of the Karp–Rabin pattern matching method [7]. It puts forward the relevance of fast convolution in the context of generalized pattern matching. In Section 5, inspired by the above algorithm, we describe a simple method for finding all possible path lengths between two landing states. The algorithm has $O(n \log n)$ complexity and is deterministic. We prove its optimality and give some examples. In Section 6 we discuss ways of generating transition sequences of a given length. We provide a simple algorithm for enumerating those sequences and a formula for counting them, that could be used to generate a transition sequence uniformly at random. We conclude with some open problems.

2. Preliminaries

The content of this section is the essential background material on mathematics of juggling.

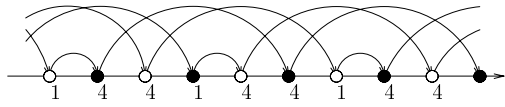


Fig. 1. Representation of a juggling sequence on the time axis.

2.1. Siteswaps

Siteswap patterns (or just “siteswaps”) allow jugglers to communicate their tricks without actually having to show them. The juggling patterns that can be represented with this notation are those in which:

- two hands are used alternately,
- two objects (say, balls) cannot be thrown at the same time,
- two objects cannot be caught at the same time.

Furthermore, the notation does not characterize the movements of the hands (crossings, for instance), nor does it provide any precise information on the trajectories of the balls in the air.

In Figure 1 we represent throws and catches on a time axis, and label them according to whether they are performed by the right or left hand. Note that catching and throwing a ball is modeled as an atomic action. In the figure we represented the trajectories in time of the balls. Since a ball cannot be sent in the past, all trajectories are hopping forward from throws to throws. The number of distinct trajectories (*orbits*) is equal to the number of balls b , which must be finite.

The *height* of a throw is the number of throws occurring between that throw and the next throw of the same ball. A (infinite) juggling sequence is nothing more than the sequence of heights. It can be formalized as a bijection $f: \mathbb{Z} \rightarrow \mathbb{Z}$ such that $f(x) > x$ (see [2]). In Figure 1 each throw is labeled with its height, showing the portion of the sequence being juggled. Note that odd heights correspond to a ball thrown at the other hand, while even heights correspond to balls that are thrown at the current hand. Empty throws are possible if no ball is scheduled to land at the current time: they are denoted by **0**.

What is commonly referred to as siteswaps are simply periodic juggling sequences. Clearly, not every sequence of positive integers can be a siteswap, since the restrictions above must hold. The sequence depicted in Figure 1 actually corresponds to the siteswap **441**. The simplest patterns are those in which the height is constant. They can be represented by a single number. The siteswap **3**, for instance, represents the classical three-ball cascade in which each ball is thrown alternately to the other hand. We can realize that the height is equal to the number of balls, since two balls must be thrown at height 3 between two throws of a given ball. Similarly, the siteswaps **4**, **5**, and **6**, etc., represent respectively the standard four-, five- and six-ball cascades. There are many ways to generate siteswaps, either exhaustively or randomly [12]. Examples of valid siteswaps include **423**, **522**, or **441** (three balls), **534** (four balls), **75**, **756**, or **774** (six balls). An example of invalid siteswap is **324**, as the first two throws will arrive at the same time, which violates the third condition on siteswaps.

A nice property of siteswaps, called the *average theorem*, is that the sum of the throw heights divided by the pattern length is equal to the number of balls b . This is true in particular, as we have seen, when the siteswap has length 1. We can compute the number of balls in the example of Figure 1: $(4 + 4 + 1)/3 = 3$. A rigorous proof of the theorem is provided in [2] and reproduced in [12]. Note that the average theorem yields a necessary condition for a siteswap to be valid, however, as witnessed by the siteswap **324**, this condition is not sufficient. A necessary and sufficient condition is as follows: $s_0 s_1 \cdots s_{p-1}$ is a valid siteswap if and only if the mapping $i \mapsto (i + s_i) \bmod p$ is a permutation of $\{0, 1, \dots, p - 1\}$.

The word siteswap comes from the *siteswapping* operation, that consists in swapping the landing times of two balls in a given siteswap in order to generate a new valid siteswap. Swapping for instance the landing times of the first **4** throw and the **1** throw in the pattern **441** leads to a new, valid pattern **342**.

2.2. Landing State Graph

Let us freeze the action described on Figure 1 at some point t in time, and represent the **expected landing times of the balls currently in the air**. This is shown at the top of Figure 2, where we froze just before the **1** throw. A ball is planned to land immediately before time $t + 1$, and two more before times $t + 3$ and $t + 4$, respectively. We do not distinguish the different balls, and simply denote by a “1” an expected landing, and by a “0” the absence of any landing. The **bit vector obtained this way is called the landing state at time t** . At the top of Figure 2, the landing state is 1011.

Let us now move one step forward in time, at time $t + 1$, and see what we can do in the meanwhile with the landing ball. Clearly, it must be thrown to some point in the future, and it cannot be sent in the future at a time when a landing is expected. In Figure 2 we show how the ball can be sent to the next beat using a **1** throw, following the **441** siteswap. The new landing state is 111.

So a landing state for b balls is simply a bit vector in which exactly b bits are set to “1”. It can be thought of as followed by an infinity of zeros but we use the convention to represent landing states up to the last “1” bit.

A throw acts on a landing state as a left shift if the leftmost bit of the state is “0”, and otherwise as a combination of the left shift and setting a “0”-bit to “1”. This is exactly how an edge is defined in the landing state graph $\mathcal{G}_b$.

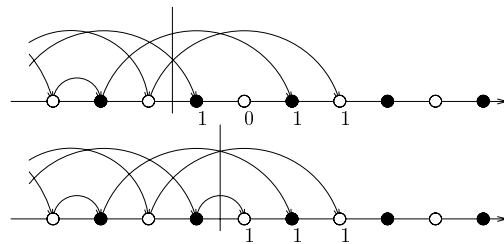


Fig. 2. Landing states.

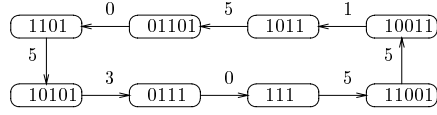


Fig. 3. The cycle in $\mathcal{G}_3$ corresponding to the three-ball siteswap **55150530**.

Definition 1. The b -ball landing state graph $\mathcal{G}_b$ is a directed graph with the set of states $(0^*1)^b$ as the vertex set and in which there exists an edge from state $X = x_1x_2 \cdots x_n$ to state Y if and only if Y can be obtained by modifying X following Algorithm 1. Each edge of $\mathcal{G}_b$ is labeled with the height of the corresponding throw, equal to **0** if $c = 0$, or to the index of the new “1” bit otherwise.

Algorithm 1. Transition in the landing state graph

```

c ← x1
shift X left by one position
if c = 1 then
  either
    choose i such that xi = 0
    xi ← 1
  or
    concatenate X with a string of the form 0*1
end if

```

Landing state graphs are usually partly represented, according to a given maximum throw height. A number of typeset landing state graphs with bounded throw height are given by Lundmark [8].

From the definition of a landing state graph, a siteswap is nothing but the sequence of edge labels in any loop (closed path) of $\mathcal{G}_b$. In Figure 3 we represented the loop in $\mathcal{G}_3$ corresponding to a long siteswap. In that case the loop is a cycle, or a *prime loop* in the juggling terminology.

3. Finding Transitions

Finding transitions between siteswap patterns is a challenging problem that occurs frequently when designing juggling routines [10].

Since a siteswap pattern is a loop in the state graph, finding a transition between two patterns—if we do not consider circular rotations of a pattern as equivalent—amounts to finding a path between two given states in the graph. The number of edges (throws) in the path cannot be chosen arbitrarily: it might be impossible to reach a state from another in a given number of throws.

Problem 1. Find all achievable path lengths between two landing states X and Y .

Definition 2. A *transition sequence* between two states X and Y is the sequence of edge labels (throw heights) on a path from X to Y in $\mathcal{G}_b$.

We now show that finding achievable path lengths between two landing states can be seen as a special kind of pattern matching problem. Let us, for instance, compute the shortest path from state $X = 1001101$ to state $Y = 1010101$. From the following alignment of X and Y , it is impossible to reach Y from X in a single throw, because some bits in Y are set to “0” while the corresponding bits in X are set to “1”:

$$\begin{array}{l} X = 1001101, \\ Y = 1010101. \end{array}$$

If we test with a shift of two positions, we obtain:

$$\begin{array}{l} X = 1001101, \\ Y = 1010101, \end{array}$$

which is still impossible. The first shift that corresponds to feasible transition sequences is the 4-position shift:

$$\begin{array}{l} X = 1001101, \\ Y = 1010101. \end{array}$$

There exists a feasible transition sequence because the two “1”s in the unmatched prefix of X can be used to fill the two “1” positions in the unmatched suffix of Y . Hence a shortest path between X and Y has length 4 and a corresponding transition sequence may be for instance **8007**. Checking all possible shifts yields a simple quadratic algorithm for finding the shortest path between X and Y .

This matching procedure can actually be seen as an instance of a “pattern matching with don’t cares” (PMWDC) problem [4], [6], [11]. In a PMWDC problem, some symbols in the pattern are “don’t care” symbols, denoted by “*”, that match any symbol in the text.

For this, transform Y as follows:

1. pad Y , if needed, by a sequence of zeros so that $|Y| \geq |X|$,
2. replace each “1” by a “*.”

Let Y' be the new string. Now any (possibly right-hanging) match of this new string with some position in X defines a path from X to Y in $\mathcal{G}_b$. The shortest path length is the position of the first match from left to right.

The above proposition can be shown to be true simply by noticing that only zeros in Y force a “0” as the matched symbol in X , while a “1” in Y can be matched with either a “0” or a “1” in X . Also, when a match occurs between Y' and X , there are always enough ones in the unmatched prefix of X to fill the unmatched ones in Y .

Note that we can find a simple upper bound on the shortest path length by trying to match exactly a suffix of state X with a prefix of state Y . This can be done in linear time using a standard pattern matching algorithm. In the example above, this upper bound appears to be optimal.

4. Pattern Matching with Don't Cares

We briefly recall a recent randomized algorithm from Kalai [6] for PMWDC. This algorithm is inspired by the well-known fingerprint method of Karp and Rabin [7] for standard pattern matching. It runs in $O(n \log m)$ time on a RAM machine, where $n = |X|$ and $m = |Y|$. It returns all matches, as well as, with an arbitrarily small probability, some false matches.

The algorithm receives $X = x_1x_2 \cdots x_n$ and $Y = y_1y_2 \cdots y_m$ as input, and the probability of a false match is controlled by a parameter N . It is given as Algorithm 2. The crucial step of the algorithm consists of a convolution between the text and a fingerprint r . The don't care symbols are taken into account in the construction of the fingerprint by letting the factor in the sequence r be equal to zero.

Clearly, a match cannot be missed. The probability that a false match occurs at a given position is the probability that $s(j) = t$ has a solution given that there is at least one wrong symbol in the text, which is at most $1/N$. The complexity of the algorithm is essentially that of the convolution, which can be computed in time $O(n \log m)$ using the Fast Fourier Transform (FFT).

Algorithm 2. PMWDC

```

for  $1 \leq i \leq m$ 
  if  $y_i = *$  then
     $r_i \leftarrow 0$ 
  else
     $r_i \leftarrow$  random from  $\{1, 2, \dots, N\}$ 
  end if
end for
 $t \leftarrow \sum_{i=1}^m y_i r_i$ 
compute  $s(j) = \sum_{i=1}^m x_{i+j} r_i$  for  $0 \leq j \leq n - m$  using FFT
output  $\{j: s(j) = t\}$ 

```

5. Algorithm and Examples

We now present a simple algorithm for solving Problem 1, inspired by the PMWDC method of the previous section. If we apply the method blindly, we realize that t can only be zero, hence that any nonzero value of $s(j)$ will not be a match. Also, zero values of $s(j)$ can only be matches and the values of r_i for $y_i = 0$ has no importance.

We can therefore let $r_i = 1$ whenever $y_i = 0$, and $r_i = 0$ whenever $y_i = 1$. The algorithm reduces to the convolution of X with the complement $\bar{Y}$ of Y , as described in Algorithm 3. It is deterministic and has complexity $O(n \log n)$, where $n = \max\{|X|, |Y|\}$.

Algorithm 3. Algorithm for Problem 1

```

pad  $Y$  with zeros if necessary so that  $|Y| \geq |X|$ 
pad  $X$  with  $|Y|$  zeros
compute  $s(j) = \sum_{i=1}^{|Y|} x_{i+j} \bar{y}_i$  for  $0 \leq j \leq |X| - |Y|$  using FFT
output  $\{j: j > 0 \text{ and } s(j) = 0\}$ 

```

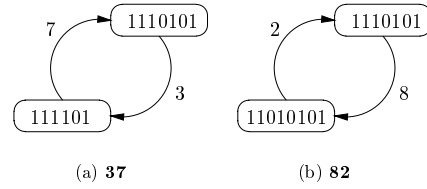


Fig. 4. Shortest cycles going through state 1110101.

Algorithm 3 returns all matches, i.e., all possible transition lengths up to the trivial case where the unmatched prefix of X contains b ones.

An interesting special case of the problem occurs when $X = Y$. In that case, our procedure finds the lengths of the loops going through the given state. The first match corresponds to the shortest cycle. Let us see for instance what happens when $X = Y = 1110101$, with $b = 5$. The first match is as follows:

$$\begin{array}{l} X = 1110101, \\ Y = \quad 1110101. \end{array}$$

We conclude that the two shortest cycles going through this state have length 2, and are respectively **82** and **37**. The intermediate state in both cases is given in Figure 4.

The complexity of the algorithm matches that of the best known algorithm for boolean convolution. Note that a number of other pattern matching problems can be shown to be harder than computing a boolean convolution [5].

Theorem 1. *If we assume that a boolean convolution cannot be computed in less than $O(n \log n)$ time, then Algorithm 3 is an optimal algorithm to solve Problem 1.*

Proof. What we actually want to compute is the boolean convolution of two binary words X and $\bar{Y}$, where the number of ones in X is equal to the number of zeros in $\bar{Y}$. We show that if we can compute this in time less than $O(n \log n)$ then we can also compute any boolean convolution in time less than $O(n \log n)$.

To show this, we encode an instance of the boolean convolution problem, made of two binary words A and B , into an instance $(X, \bar{Y})$ of the restricted problem as follows. If the number of ones in A is greater than the number of zeros in B , we can let $\bar{Y}$ be equal to the concatenation of B with a sufficient number of zeros, and let X be equal to A . This clearly does not change the result of the convolution. On the other hand, if the number of ones in A is less than the number of zeros in B , we can let X be equal to A concatenated with $|B|$ zeros and a sufficient number of ones, and let $\bar{Y}$ be equal to B . The result of the original convolution is a prefix of the convolution of X and $\bar{Y}$. $\square$

We conclude this section by mentioning that in practice, PMWDC can be implemented efficiently using the Shift-Or algorithm from Baeza-Yates and Gonnet [1]. This algorithm uses bitwise techniques and relies on the fact that the pattern is not much longer than a memory word. The algorithm is in fact a dynamic programming algorithm in which a whole column of the dynamic programming table can be stored in a single

memory word and updated with a couple of bitwise operations. The algorithm can be used for solving Problem 1 in cases where the landing states can be encoded in a small number of bits, hence if the throw heights and the number of balls are small.

6. Enumerating Transition Sequences

Now that we have an algorithm for finding the lengths of the paths leading from landing state X to Y , we may ask ourselves what can be the transition sequences for these lengths.

Problem 2. Given an achievable path length ℓ , output all transition sequences of length ℓ .

We first remark that the balls corresponding to the “1”s in the unmatched prefix of X need not necessarily be thrown at their final position in the future. They may as well be thrown multiple times before landing in their final position. As an illustration, consider two distinct transition sequences corresponding to the following alignment of two landing states with $b = 4$:

$$\begin{array}{rcl} X & = & 1010101, \\ Y & = & 1111. \end{array}$$

The first obvious transition sequence is the one sending the first ball in the first position to fill in the target state, and the second in the second such position. This is the sequence **5050** illustrated in Figure 5(a). For coherence, **0**-throws should be seen as pictured: an instantaneous throw from one hand to itself. Now let us apply siteswapping and swap the landing positions of some of the throws to obtain a new valid sequence of throws.

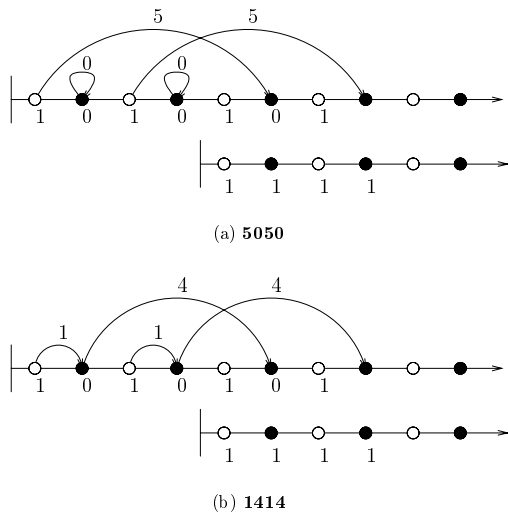


Fig. 5. Two transition sequences of length 4 between states 1010101 and 1111.

If we swap the landing positions of the first two throws, and then swap the landing positions of the two next throws, we obtain another valid transition sequence: **1414**, shown in Figure 5(b). In fact, we can jump from any transition sequence to any other by siteswapping, just as for regular siteswap patterns [12].

From this description, it is a simple matter to enumerate all possible transition sequences: we choose the landing time of the first ball and recurse. The landing time can be chosen among the times corresponding to all zeros in the unmatched prefix of X , and all zeros in X that are matched with ones in Y . The new landing state obtained after this throw is made is still, by definition, correctly matched with Y . We can show that the sum of the heights must be constant (it is preserved by siteswapping), but that the average theorem is not true anymore.

We also give the number of transition sequences of a given length. This number depends only on the unmatched prefix of X .

Theorem 2. *If ℓ is an achievable path length between X and Y , then the number of distinct transition sequences (paths) of length ℓ is equal to*

$$k! \prod_{i=1}^k (i+1)^{n_i}, \quad (1)$$

where:

- k is the number of ones in the ℓ -prefix of X ,
- n_i is the number of zeros between the i th and the $(i+1)$ th “1” in the ℓ -prefix of X if $0 < i < k$, or the number of zeros between the k th “1” and position $\ell+1$ if $i = k$.

Proof. First, we explain the $k!$ factor in front of the formula. The number k is the number of ones in the ℓ -prefix of X . These k ones will have to fill k positions with indices in X greater than ℓ . Any of the k balls can fill those k positions, so there are $k!$ combinations corresponding to the possible final positions. From now on, we consider that a throw either sends a ball in its final position after ℓ , or sends a ball in some position before ℓ , and that ball should be thrown again later.

We define $F(n_1, n_2, \dots, n_k)$ as the number of transition sequences, divided by the $k!$ factor. This is the number of transition sequences if we assume that each ball has its final position in Y already assigned.

In the i th run of zeros, with length n_i , each “0” position can either be assigned one of the i balls corresponding to the first i ones in the prefix, or can be left empty. So for each such “0”, we have $i+1$ possibilities. These possibilities can be seen to be independent, so the number $F(n_1, n_2, \dots, n_k)$ is the product of the number of words of size n_i that can be constructed with the alphabet $1, 2, \dots, i+1$, for each i between 1 and k . This concludes the proof of Theorem 2. $\square$

We provide a more formal proof of this proposition in the Appendix.

As an example, for $X = 1010101$, $Y = 1111$, and $\ell = 4$, we have $k = 2$, $n_1 = 1$, and $n_2 = 1$. The number of transition sequences of length 4 between X and Y is therefore

equal to $2!2^13^1 = 12$. The complete list is: **1234, 1252, 1414, 1450, 1612, 1630, 3034, 3052, 5014, 5050, 7012, 7030**.

7. Conclusion

An interesting open question, which is important in practice, is whether we can find quickly the set of transition lengths and sequences between two states if we restrict the maximum height of a throw. This is the shortest path problem in the finite directed landing state graph $\mathcal{G}_{b,h}$, where b is the number of balls and h is the maximum throw height.

Acknowledgment

The authors wish to thank Jack Boyce for helpful comments on an earlier version of this paper.

Appendix. Proof of Theorem 2

Proof. We first need a technical lemma. The proof is by induction on n .

Lemma 1. *For all positive integer n ,*

$$\sum_{j=1}^n a^{j-1}(a+1)^{n-j} = (a+1)^n - a^n. \quad (2)$$

We now prove Theorem 2 by induction using a recursion formula.

We classify the transition sequences into three groups. Without loss of generality, we assume that the first throw is a nonzero throw, so we ignore all leading zeros in X .

The transition sequences in the first group are those whose first throw is between 1 and n_1 . The number of such sequences is

$$\sum_{j=1}^{n_1} F(n_1 - j, n_2, \dots, n_k). \quad (3)$$

The second group is composed of sequences beginning with a throw between $n_1 + 2$ and $\ell - 1$. The number of such sequences is

$$\sum_{i=2}^k \sum_{j=1}^{n_i} F(n_2, \dots, n_{i-1}, j-1, n_i - j, n_{i+1}, \dots, n_k). \quad (4)$$

The first sum is the choice of a run of zeros in which the ball will be sent, and the second sum is over all the possibilities for a throw in that range.

The third group is the group of sequences in which the first throw places a ball in its final position, hence whose first throw is at least ℓ . There are

$$F(n_2, n_3, \dots, n_k) \quad (5)$$

such sequences.

From these considerations, we have that

$$\begin{aligned}
 F(n_1, n_2, \dots, n_k) &= \sum_{j=1}^{n_1} F(n_1 - j, n_2, \dots, n_k) \\
 &\quad + \sum_{i=2}^k \sum_{j=1}^{n_i} F(n_2, \dots, n_{i-1}, j-1, n_i - j, n_{i+1}, \dots, n_k) \\
 &\quad + F(n_2, n_3, \dots, n_k),
 \end{aligned} \tag{6}$$

with initial conditions of the form $F(0, 0, \dots, 0) = 1$. We now show by induction that the solution is $F(n_1, n_2, \dots, n_k) = \prod_{i=1}^k (i+1)^{n_i}$.

Let us assume that the result is correct for all vectors $n'_1, n'_2, \dots, n'_{k'}$ with $k' \leq k$ and whose sum is less than $\sum_{i=1}^k n_i$. Then we can replace the values of F in the recursion formula:

$$\begin{aligned}
 F(n_1, n_2, \dots, n_k) &= \sum_{j=1}^{n_1} \left(2^{n_1-j} \prod_{i=2}^k (i+1)^{n_i} \right) \\
 &\quad + \sum_{i=2}^k \sum_{j=1}^{n_i} \left(\prod_{t=2}^{i-1} t^{n_t} \right) i^{j-1} (i+1)^{n_i-j} \left(\prod_{t=i+1}^k (t+1)^{n_t} \right) + \prod_{i=2}^k i^{n_i}
 \end{aligned} \tag{7}$$

$$\begin{aligned}
 &= (2^{n_1} - 1) \prod_{i=2}^k (i+1)^{n_i} \\
 &\quad + \sum_{i=2}^k \left(\prod_{t=2}^{i-1} t^{n_t} \right) \left(\prod_{t=i+1}^k (t+1)^{n_t} \right) \left(\sum_{j=1}^{n_i} i^{j-1} (i+1)^{n_i-j} \right) + \prod_{i=2}^k i^{n_i}.
 \end{aligned} \tag{8}$$

Now from Lemma 1, we have $\sum_{j=1}^{n_i} i^{j-1} (i+1)^{n_i-j} = (i+1)^{n_i} - i^{n_i}$.

If we substitute this expression in the sum above, we can delete terms in a pairwise fashion, and show that

$$\begin{aligned}
 &\sum_{i=2}^k \left(\prod_{t=2}^{i-1} t^{n_t} \right) \left(\prod_{t=i+1}^k (t+1)^{n_t} \right) \left(\sum_{j=1}^{n_i} i^{j-1} (i+1)^{n_i-j} \right) \\
 &= \sum_{i=2}^k \left(\prod_{t=2}^{i-1} t^{n_t} \right) \left(\prod_{t=i+1}^k (t+1)^{n_t} \right) ((i+1)^{n_i} - i^{n_i})
 \end{aligned} \tag{9}$$

$$= \sum_{i=2}^k \prod_{t=2}^{i-1} t^{n_t} \prod_{t=i}^k (t+1)^{n_t} - \sum_{i=2}^k \prod_{t=2}^i t^{n_t} \prod_{t=i+1}^k (t+1)^{n_t} \tag{10}$$

$$= \prod_{t=2}^k (t+1)^{n_t} - \prod_{t=2}^k t^{n_t}. \tag{11}$$

Substituting again in (8), we obtain the announced result:

$$F(n_1, n_2, \dots, n_k) = (2^{n_1} - 1) \prod_{i=2}^k (i+1)^{n_i} + \prod_{i=2}^k (i+1)^{n_i} - \prod_{i=2}^k i^{n_i} + \prod_{i=2}^k i^{n_i} \quad (12)$$

$$= \prod_{i=1}^k (i+1)^{n_i}. \quad \square \quad (13)$$

References

- [1] R.A. Baeza-Yates and G.H. Gonnet. A new approach to text searching. *Communications of the ACM*, 35(10):74–82, 1992.
- [2] J. Buhler, D. Eisenbud, R. Graham, and C. Wright. Juggling drops and descents. *Amer. Math. Monthly*, 101:507–519, 1994.
- [3] R. Ehrenborg and M. Readdy. Juggling and applications to q -analogues. *Discrete Math.*, 157:107–125, 1996.
- [4] M. Fischer and M. Paterson. *Complexity of Computation*, pages 113–125. SIAM–AMS, Providence, RI, 1974. (SIAM–AMS Proceedings 7, R.M. Karp, editor)
- [5] P. Indyk. Faster algorithms for string matching problems: matching the convolution bound. In *Proceedings of the 39th Symposium on Foundations of Computer Science*, pages 166–173, 1998.
- [6] A. Kalai. Efficient pattern-matching with don't cares. In *Proceedings of the Thirteenth Annual ACM–SIAM Symposium on Discrete Algorithms*, pages 655–656, 2002.
- [7] R.M. Karp and M.O. Rabin. Efficient randomized pattern-matching algorithms. *IBM J. Res. Develop.*, 31(2):249–260, 1987.
- [8] H. Lundmark. Siteswap state diagrams. <http://www.mai.liu.se/~halun/>, 2003.
- [9] B. Magnusson and B. Tiemann. A notation for juggling tricks. *Juggler's World*, pages 31–33, summer 1991.
- [10] C. Miller. Finding juggling pattern transitions with smart engineering systems. <http://www.juggling.org/papers/miller/>, 1997.
- [11] S. Muthukrishnan and K. Palem. Non-standard stringology: algorithms and complexity. In *Proceedings of the 26th Symposium on the Theory of Computing*, pages 770–779, 1994.
- [12] B. Polster. *The Mathematics of Juggling*. Springer-Verlag, New York, 2003.
- [13] C.E. Shannon. Claude Shannon's no-drop juggling diorama. In N. Sloane and A. Wyner, editors, *Claude Elwood Shannon – Collected Papers*, pages 847–849. IEEE Press, Piscataway, NJ, 1993.
- [14] C.E. Shannon. Scientific aspects of juggling. In N. Sloane and A. Wyner, editors, *Claude Elwood Shannon – Collected Papers*, pages 850–864. IEEE Press, Piscataway, NJ, 1993.
- [15] C. Simpson. Juggling on paper. *Juggler's World*, page 31, winter 1986.
- [16] E. Steingrimsdottir. Permutation Statistics of Indexed and Poset Permutations. Ph.D. thesis, MIT, Cambridge, MA, 1991.

Received July 6, 2004, and in final form May 25, 2005. Online publication March 3, 2006.